W02 - Compilation on HPC

Mohammad Amin GHAVAMI

Uni.lu HPC username: "mghavami"

GitHub username: "AminGhavami"

University of Luxembourg

March 11, 2025

1 Question 2:

Check the commands `cc` and `gcc`:

- What is the type of these commands? What is their full path?
  They are files. `cc` is located: `/usr/bin/cc` and `gcc` is located: `/usr/bin/gcc`
- What is the name of these compilers? What version?
  `cc / gcc (GCC) 8.5.0 20210514 (Red Hat 8.5.0-22)`
- Are they the same compiler? Why?
  yes, because `cc` is linked to `gcc`

```
0 [mghavami@aion-0219 ~](6508571 1N/1T/1CN)$ which cc
/usr/bin/cc
0 [mghavami@aion-0219 ~](6508571 1N/1T/1CN)$ type cc
cc is /usr/bin/cc
0 [mghavami@aion-0219 ~](6508571 1N/1T/1CN)$ file cc
cc: cannot open `cc' (No such file or directory)
0 [mghavami@aion-0219 ~](6508571 1N/1T/1CN)$ ls -l /usr/bin/cc
lrwxrwxrwx 1 root root 3 Apr 19  2024 /usr/bin/cc -> gcc
0 [mghavami@aion-0219 ~](6508571 1N/1T/1CN)$ file /usr/bin/cc
/usr/bin/cc: symbolic link to gcc
0 [mghavami@aion-0219 ~](6508571 1N/1T/1CN)$ which gcc
/usr/bin/gcc
0 [mghavami@aion-0219 ~](6508571 1N/1T/1CN)$ type gcc
gcc is /usr/bin/gcc
0 [mghavami@aion-0219 ~](6508571 1N/1T/1CN)$ file /usr/bin/gcc
/usr/bin/gcc: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked, i
nterpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, BuildID[sha1]=036f151cce4017d2
4eac7b499a6e9db67bf9200e, stripped
0 [mghavami@aion-0219 ~](6508571 1N/1T/1CN)$ ls -l /usr/bin/gcc
-rwxr-xr-x 3 root root 1262600 Apr 19  2024 /usr/bin/gcc
0 [mghavami@aion-0219 ~](6508571 1N/1T/1CN)$ gcc --version
gcc (GCC) 8.5.0 20210514 (Red Hat 8.5.0-22)
Copyright (C) 2018 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

0 [mghavami@aion-0219 ~](6508571 1N/1T/1CN)$ cc --version
cc (GCC) 8.5.0 20210514 (Red Hat 8.5.0-22)
Copyright (C) 2018 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Figure 1: results related to question 2

2 Question 3:

For each of the compilers in the table above, find them on the Aion cluster and write down:

- the full module name, with class-name-version
  GNU Compiler Collection (GCC): `compiler/GCC/10.2.0`
  LVVM C, C++, Objective-C compiler: `compiler/Clang/11.0.1-1-GCCcore-10.2.0`
  Intel C, C++ Fortran compilers: `compiler/iccifort/2020.4.304`
  AMD Optimized C/C++ Fortran compilers (AOCC): `compiler/AOCC/3.1.0-GCCcore-10.2.0`

- the full path of the C compiler
  GCC: `/opt/apps/resif/aion/2020b/epyc/modules/all/compiler/GCC` Clang: `/opt/apps/resif/aion/2020b/epyc`
  iccifort: `/opt/apps/resif/aion/2020b/epyc/modules/all/compiler/iccifort` AOCC: `/opt/apps/resif/aion/20`
- the version returned by the C compiler executable

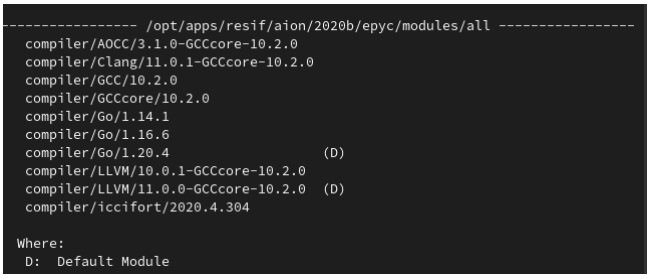

Figure 2: third question

3 Question 4:

List the main components of the main toolchains:

- For `toolchain/foss/2020b`: the module name and version of the compiler, `MPI, BLAS, LAPACK` and `FFTW`

  compiler: `GCC/10.2.0`
  MPI: `OpenMPI/4.0.5-GCC-10.2.0`
  BLAS: `numlib/OpenBLAS/0.3.12-GCC-10.2.0`
  LAPACK: `numlib/ScaLAPACK/2.1.2-gompi-2020`
  FFTW: `numlib/FFTW/3.3.8-gompi-2020b`

- For `toolchain/intel/2020b`: the module **name** and **version** of the compiler, `MPI`, and `MKL`.

  compiler: `GCCcore/10.2.0` and `iccifort/2020.4.304`
  MPI: `impi/2019.9.304-iccifort-2020.4.304`
  MKL: `numlib/imkl/2020.4.304-iimpi-2020b`

On the Uni.lu HPC, the modules are named `<class>/<name>/<version>`.

4 Question 5:

Compile the "Hello World!" program with the FOSS toolchain: Follow the five steps above and add a screenshot to your report

```
0 [mghavami@aion-0246 hello_world](6536264 1N/T/1CN)$ module load toolchain/foss/2020b
0 [mghavami@aion-0246 hello_world](6536264 1N/T/1CN)$ g++ hello.cpp -o hello
0 [mghavami@aion-0246 hello_world](6536264 1N/T/1CN)$ ./hello
Hello, World!
0 [mghavami@aion-0246 hello_world](6536264 1N/T/1CN)$ mpicxx hello_mpi.cpp -o hello_mpi
0 [mghavami@aion-0246 hello_world](6536264 1N/T/1CN)$ ./hello_mpi
Hello, World, I am 0 of 1 on aion-0246
```

Figure 3: Answers to the fifth question

5 Question 6:

Compile the "Hello World!" program with the Intel toolchain: Adapt the five steps above and add a screenshot to your report

```
0 [mghavami@aion-0246 hello_world](6536264 1N/T/1CN)$ module load toolchain/intel/2020b
0 [mghavami@aion-0246 hello_world](6536264 1N/T/1CN)$ icpc hello.cpp -o hello
0 [mghavami@aion-0246 hello_world](6536264 1N/T/1CN)$ ./hello
Hello, World!
0 [mghavami@aion-0246 hello_world](6536264 1N/T/1CN)$ mpiicpc hello_mpi.cpp -o hello_mpi
0 [mghavami@aion-0246 hello_world](6536264 1N/T/1CN)$ ./hello_mpi
Hello, World, I am 0 of 1 on aion-0246
0 [mghavami@aion-0246 hello_world](6536264 1N/T/1CN)$
```

Figure 4: Answers to the question 6

6 Question 7:

Fill the missing holes in the installation procedure for the MPFR library:

- write down the complete working installation procedure in your report

```
# Use a dedicated work directory
mkdir mpfr  #already exists
cd mpfr

# Download the source code
wget https://www.mpfr.org/mpfr-4.2.1/mpfr-4.2.1.tar.gz

# Extract the source code
tar -xzf mpfr-4.2.1.tar.gz

# Create a build directory
mkdir build
cd build

# Load the required modules
module load compiler/GCC/10.2.0
module load math/GMP/6.2.0-GCCcore-10.2.0

# Set the environment variables for the compilation options
export CFLAGS="-O3 -march=native -mtune=native -fPIC -pthread"
export CXXFLAGS="$CFLAGS"
export LDFLAGS="-pthread"

# Configure the build
# - set the install path
# - set the configure option
../configure --prefix=$HOME/mpfr-install --with-gmp=$(dirname $(dirname $(which gcc))
    ) --enable-thread-safe CFLAGS="-O3 -march=native -mtune=native -fPIC -pthread"


# Build the library (and save output)
make -j 8 |& tee build.log

# Install the library (and save output)
make install |& tee install.log
```

- add the files `config.log`, `build.log` and `install.log` (and only these files) to your GitHub classroom repository in the `mpfr` directory
  Done!

7 Question 8:

Fill the missing holes in the installation procedure for the preCICE library:

- write down the complete working installation procedure in your report

```
# Use a dedicated work directory
mkdir precise   #already exists
cd precice

# Download the source code
wget https://github.com/precice/precice/archive/refs/tags/v3.1.2.tar.gz

```

```
    # Extract the source code
    tar -xzf v3.1.2.tar.gz
    # Create a build directory
    mkdir build
    cd build

    # Load the required modules
    module load mpi/OpenMPI/4.0.5-GCC-10.2.0
    module load devel/CMake/3.20.1-GCCcore-10.2.0
    module load numlib/PETSc/3.14.4-foss-2020b
    module load lang/Python/3.8.6-GCCcore-10.2.0
    module load math/Eigen/3.4.0-GCCcore-10.2.0



    # Set the environment variables for the compilation options
    export CFLAGS="-march=native -O3"
    export CXXFLAGS="-march=native -O3"
    export FCFLAGS="-march=native -O3"


    # Configure the build
    # - set the install path
    # - set the build type
    # - set the cmake option to enable MPI communications, Python actions, PETSc mapping,
        C and Fortran bindings

    cmake ..   -DCMAKE_INSTALL_PREFIX=$HOME/precice-install   -DCMAKE_BUILD_TYPE=Release
          -DPRECICE_ENABLE_MPI_COMM=ON   -DPRECICE_ENABLE_PYTHON_ACTIONS=ON   -
        DPRECICE_ENABLE_PETSC_MAPPING=ON   -DPRECICE_ENABLE_C_BINDINGS=ON   -
        DPRECICE_ENABLE_FORTRAN_BINDINGS=ON |& tee cmake.log


    # Build the library (and save output)
    make -j 16 |& tee build.log

    # Install the library (and save output)
    make install |& tee install.log
```

- add the files `cmake.log`, `build.log` and `install.log` (and only these files) to your GitHub classroom repository in the `precice` directory
  Done!

8 Question 9:

Compile and run HPL with the instructions above.

- add the configuration file `Make.Aion_Intel64` in the `hpl` directory of your GitHub classroom repository
  Done!
- save the output of the HPL execution and add it to the `hpl` directory your GitHub classroom repository
  Done!

  Executed procedure:

```
    # Download the source code
    wget http://www.netlib.org/benchmark/hpl/hpl-2.3.tar.gz

    # Extract the source code
    tar -xzf hpl-2.3.tar.gz

    # Copy the example configuration file
    cp setup/Make.Linux_Intel64 Make.Aion_Intel64

    # Edit the configuration file
    vim Make.Aion_Intel64
        # Set variable ARCH to Aion_Intel64
        # Set variable TOPdir to the path to your top-level directory (mine was:/home/
            users/mghavami/y2425-w02-compilation-on-hpc-AminGhavami/hpl/hpl-2.3)
        # Set variable OMP_DEFS to -qopenmp
        # Add -xHost to the variable CCFLAGS
```

```
16
17      # Load Intel Toolchain
18      module load toolchain/intel/2020b
19
20      # compile HPL
21      make arch=Aion_Intel64
22
23      # Execute a short test of HPL
24      cd bin/Aion_Intel64
25      srun -n 4 -c 8 ./xhpl
```